

noted

QN-PREMISES MLOPS COCKPIT

AIRFLOW

MLFLOW

HYDRA

DVC

GIT

EVIDENTLY

MINIO

MCP

LLM

LLM

LLM PATH

LLM
PATH

EMI3 FINAL DELIVERY

António Cruz, Bruno Santos, Pedro Miranda, Ricardo Kayseller

Table of Contents

1. Introduction and Objectives	4
1.1 Problem Statement.....	4
1.2 The noted Platform	4
1.3 Project Scope Alignment	4
2. Infrastructure and Containerization.....	5
2.1 Docker Compose Architecture	5
2.2 Host Directory Mounts	5
2.3 GPU Support	6
3. Reproducibility and Configuration Management.....	6
3.1 Reproducibility Guarantees	6
3.2 Hydra Configuration.....	6
4. Pipeline Orchestration (Apache Airflow)	7
4.1 DAG Architecture.....	7
4.2 Configuration Integration	8
4.3 Modular Source Code	8
5. Experiment Tracking and Model Registry	8
5.1 MLflow Integration	8
5.2 Experiment Results.....	9
5.3 Automated Model Registration and Promotion.....	9
6. Model Serving and Frontend	9
6.1 FastAPI Serving Layer	9
6.2 noted “Try It” Panel.....	10
6.3 Jena Client Web Application	10
7. AI-Powered Development Assistant	10
7.1 Dual-Backend Architecture	10
7.2 MCP Server (Model Context Protocol)	10
7.3 Tool System	11
7.4 Web Content Fetch.....	11
8. Demonstration Pipeline: End-to-End Workflow	11
Step 1: Configuration.....	11
Step 2: Pipeline Trigger	11
Step 3: Live Monitoring	11
Step 4: Verification	11
Step 5: Auto-Promotion	12
Step 6: Prediction.....	12

Step 7: Evidently Reports	12
9. Conclusion.....	12
9.1 Summary	12
9.2 Future Enhancements.....	13

1. Introduction and Objectives

1.1 Problem Statement

Building machine learning models requires operating across a fragmented landscape of tools, each with its own interface and mental model. Notebooks serve interactive exploration but lack built-in versioning or tracking. MLflow tracks experiments but requires switching to a separate UI. DVC versions data but lives in the terminal. Hydra manages configurations but through scattered YAML files. Airflow orchestrates pipelines but demands yet another dashboard. Practitioners spend significant time context-switching between browser tabs, terminals, and dashboards, and configuration drift between experimentation and deployment is a persistent source of production failures.

1.2 The noted Platform

noted is an integrated, web-based MLOps platform that unifies interactive notebooks, data versioning, experiment tracking, configuration management, pipeline orchestration, model governance, and model serving into a single collaborative interface. The underlying tools - MLflow, DVC, Hydra, Apache Airflow, MinIO - remain the engines. noted is the cockpit.

The platform enforces a zero vendor lock-in principle: every artifact noted creates works without noted. Notebooks are standard .ipynb. MLflow runs are standard MLflow runs. DVC tracking uses standard .dvc files. Hydra configs are standard YAML. Airflow DAGs use only standard operators. If noted is removed, the entire MLOps stack continues to run independently.

1.3 Project Scope Alignment

This report demonstrates that the Jena Weather Forecasting project, built on the noted platform, fulfills all final delivery requirements:

Requirement	Implementation
Automated Airflow pipeline with model registration	6-task DAG with auto-promote to MLflow Registry
FastAPI serving layer with dynamic model loading	noted-serving container loads @champion from registry
Functional frontend for real-time predictions	jena_client web app (FastAPI + socket.io + Chart.js)
End-to-end demonstration pipeline	Hydra config -> Airflow trigger -> MLflow verification -> API query
Docker Compose containerization	12+ containers with GPU support
Hydra configuration management	4 config groups (data, model, training, scaler)
DVC data versioning	Jena Climate dataset tracked with MinIO remote
100% reproducibility	DVC hash + Hydra hash + git commit + seed = identical results

2. Infrastructure and Containerization

2.1 Docker Compose Architecture

The platform runs as a multi-container deployment orchestrated via Docker Compose. The architecture separates concerns across 12+ services, each with a single responsibility:

Service	Container	Purpose
noted	noted	FastAPI backend + static frontend (main platform)
MLflow	noted-mlflow	Experiment tracking + model registry
Airflow API Server	noted-airflow-apiserver	Pipeline REST API (Airflow 3.0)
Airflow Scheduler	noted-airflow-scheduler	DAG scheduling
Airflow Worker	noted-airflow-worker	Celery task execution (GPU-enabled)
MinIO	noted-minio	S3-compatible object storage (DVC remote + MLflow artifacts)
PostgreSQL	noted-postgres	Shared metadata (MLflow + Airflow)
Redis	noted-redis	Airflow Celery broker
Model Serving	noted-serving	FastAPI model inference
Evidently	noted-evidently	Data quality and drift monitoring
Agent Server	agent_server	Local LLM inference (Gemma 4)
nginx	noted-nginx	Reverse proxy

All services share a Docker network. The noted container acts as a single proxy to all backend services - secrets (API keys, database credentials) are managed server-side via Infisical and never reach the browser.

2.2 Host Directory Mounts

External projects (like the Jena Weather project) are linked into noted via host directory mounts configured in data/NOTED.md. On startup, noted auto-generates docker-compose.mounts.yml with volume entries for both the noted container and all Airflow services, ensuring DAGs in project dags/ folders are automatically discovered by Airflow without manual configuration.

2.3 GPU Support

The deployment includes NVIDIA CUDA runtime support. Training tasks in the Airflow worker container have direct GPU access, enabling accelerated model training with TensorFlow, PyTorch, and other GPU-enabled frameworks.

3. Reproducibility and Configuration Management

3.1 Reproducibility Guarantees

The platform enforces 100% reproducibility through four interlocking mechanisms:

1. **Data versioning (DVC):** The Jena Climate dataset (data/jena_climate_2009_2016.csv, 41.2 MB) is tracked by DVC with MinIO as the remote storage backend. Every training run is tagged with the DVC data hash (dvc_data_hash), linking the model to the exact dataset version that produced it.
2. **Configuration hashing (Hydra):** The resolved Hydra configuration is hashed (SHA-256) and logged as both an MLflow parameter (hydra_config_hash) and tag on every experiment run. This guarantees that two runs with the same config hash used identical hyperparameters.
3. **Code versioning (Git):** All source code, configuration files, and DAG definitions are tracked in Git. Experiment snapshots capture the exact git commit, creating a branch (snapshot/{experiment}_{version}) that preserves the code state.
4. **Seed control:** Random seeds are managed via the Hydra training config (seed parameter) and propagated to TensorFlow, NumPy, and Python's random module, ensuring deterministic training when combined with the same data and configuration.

To reproduce any experiment: clone the repository, dvc pull the data, load the Hydra config from the run's hydra_config_hash, and execute the pipeline. The result will match the original MLflow metrics exactly.

3.2 Hydra Configuration

Configuration is managed hierarchically using Hydra with four config groups:

```
config/
  config.yaml          # Root config with group defaults
  data/
    default.yaml       # Dataset paths, train/val/test splits
  model/
    gru_baseline.yaml  # GRU architecture (units, dropout, layers)
    gru_evolutionary.yaml # Evolved GRU architecture
  training/
    default.yaml       # Epochs, batch size, learning rate, seed
```

```
scaler/  
  standard.yaml      # StandardScaler  
  robust.yaml        # RobustScaler  
  minmax.yaml        # MinMaxScaler
```

The noted UI provides a Configuration Composer panel where users select options from each group via dropdowns. The composed configuration is injected into notebook kernels as a `cfg` object via `json.loads()` (avoiding `null/true/false` issues with Python's `eval()`). The same YAML configs are read directly by Airflow DAG tasks, ensuring notebook experiments and pipeline runs use identical configuration resolution.

4. Pipeline Orchestration (Apache Airflow)

4.1 DAG Architecture

The Jena Weather training pipeline is implemented as an Airflow 3.0 DAG (`dags/jena_training_pipeline.py`) with six tasks:

```
ingest_data (1.5s)  
  |  
preprocess_data (0.4s)  
  |  
  +-- evidently_quality (4.8s)    [parallel branch]  
  |  
  +-- train_model_task (192.5s)  [parallel branch]  
      |  
      +-- promote_model (1.0s)  
      |  
      +-- evidently_drift (7.8s)
```

Task descriptions:

1. **ingest_data**: Loads the raw CSV from DVC-tracked path, validates schema, writes to `/tmp` parquet for downstream tasks
2. **preprocess_data**: Applies feature engineering (cyclical time features, rolling statistics), scales features using the Hydra-configured scaler, creates sliding windows
3. **evidently_quality**: Runs Evidently data quality report on the preprocessed data (parallel with training)
4. **train_model_task**: Builds and trains the GRU model per Hydra config, logs metrics and artifacts to MLflow (192.5s on GPU)
5. **promote_model**: Compares the new model against the current `@champion` in the MLflow Registry; promotes if test MAE improves
6. **evidently_drift**: Runs Evidently drift detection comparing train vs test distributions

4.2 Configuration Integration

The DAG is parameterized via Airflow DAG params that map directly to Hydra config groups. When triggered from the noted UI, the Configuration Composer panel's selections are passed as DAG parameters. The DAG tasks read the same Hydra YAML configs that notebooks use, ensuring configuration consistency across interactive and automated execution.

4.3 Modular Source Code

All pipeline logic resides in reusable src/ modules shared between the notebook and the Airflow DAG:

Module	Purpose
src/data/ingestion.py	Data loading and validation
src/data/preprocessing.py	Feature engineering and preprocessing
src/data/preparation.py	Scaling, windowing, train/val/test split
src/training/pipeline.py	Model building, training, MLflow logging
src/evaluation/metrics.py	Test evaluation metrics (MAE, RMSE, R2)
src/evaluation/promote.py	Champion comparison and auto-promotion

This design eliminates code duplication: the notebook calls the same functions that Airflow tasks execute. Changes to data processing logic automatically propagate to both execution paths.

5. Experiment Tracking and Model Registry

5.1 MLflow Integration

noted provides zero-config MLflow connectivity. The MLFLOW_TRACKING_URI is injected into every kernel automatically - import mlflow just works without any boilerplate. The platform supports two complementary tracking modes:

1. **Auto-instrumentation:** The Run Manager UI defines named cell groups as reusable run templates. Executing a run wraps the selected cells in `mlflow.start_run()/end_run()` automatically, with framework autologging activated for TensorFlow, scikit-learn, and other supported frameworks.
2. **Pipeline tracking:** Airflow DAG tasks call MLflow directly through the `src/training/pipeline.py` module, logging metrics (`test_mae`, `test_rmse`, `test_r2`), parameters (all Hydra config values), and artifacts (trained model, training history).

Both modes automatically tag runs with the DVC data hash and Hydra config hash, providing full lineage traceability.

5.2 Experiment Results

The jena_weather experiment contains 13 completed runs across two configuration variants:

Configuration	Epochs	Units	Test MAE (C)	Test R2
GRU Baseline (30 epochs, batch 256)	30	128/64	2.02	0.893
GRU Extended (50 epochs, batch 128)	50	96/64	1.68	-
Persistence Baseline	-	-	3.14	-

The GRU models significantly outperform the persistence baseline (3.14 C MAE), demonstrating the value of the deep learning approach for temperature forecasting.

5.3 Automated Model Registration and Promotion

The promote_model task in the Airflow DAG implements automatic champion selection:

1. After training completes, the new model's test MAE is compared against the current @champion model in the MLflow Registry
2. If the new model improves on the champion's metric, it is registered as a new version and the @champion alias is reassigned
3. The promotion decision is logged as an MLflow tag for audit

This ensures the serving endpoint always loads the best available model without manual intervention.

6. Model Serving and Frontend

6.1 FastAPI Serving Layer

The noted-serving container is a dedicated FastAPI service that dynamically loads any registered model from the MLflow Registry on demand. Key capabilities:

- **Dynamic model loading:** Load any model version by name and version/alias. The @champion alias provides a stable reference to the best model
- **Schema-aware input validation:** Pydantic schemas derived from the model's MLflow signature validate incoming JSON requests
- **Multi-framework support:** Pre-installed TensorFlow, PyTorch, scikit-learn, XGBoost, and LightGBM for model inference
- **Health monitoring:** /health endpoint with serving status visible in noted's status bar

6.2 noted “Try It” Panel

Within the noted platform, the Model Registry section provides a “Try It” panel for any registered model. The panel generates a dynamic input form based on the model’s schema signature, sends the request through noted’s backend (which proxies to the serving container), and renders the output as charts (ECharts), tables, or formatted JSON.

6.3 Jena Client Web Application

Beyond noted’s built-in serving interface, a standalone web application (`jena_client`) demonstrates the model serving for end users:

- **Backend:** FastAPI + socket.io server that connects to the noted-serving container
- **Frontend:** Vanilla HTML/CSS/JavaScript with Chart.js for interactive forecast visualization
- **Features:** Real-time temperature predictions, historical data overlay, dark/light theme toggle
- **Deployment:** Runs as a separate service, accessible to users who don’t need the full noted platform

This dual approach demonstrates both integrated (Try It panel) and standalone (`jena_client`) serving patterns.

7. AI-Powered Development Assistant

7.1 Dual-Backend Architecture

noted integrates an AI assistant that understands the full MLOps workspace and can both reason about it and act on it through structured tool calls. The assistant supports two inference backends:

- **Gemma 4 E4B** (local, via llama-cpp-python): On-premises inference with 128K context window. No data leaves the host. Native tool calling via trained `<|tool_call|>` special tokens
- **Anthropic Claude** (Sonnet 4.6, Opus 4.6, Haiku 4.5): Cloud API with 200K context window. Native tool calling via Anthropic’s tools array and `tool_use` content blocks

Both backends use their model-native tool calling mechanisms rather than text-based prompt injection, ensuring reliable structured arguments and eliminating parsing fragility.

7.2 MCP Server (Model Context Protocol)

noted exposes its tool surface through an MCP server at `/mcp/`, enabling external AI clients (Claude Code, Claude Desktop, Cursor) to discover and invoke noted’s capabilities without the noted UI. The server uses Streamable HTTP transport with the official mcp Python SDK, includes rate limiting (tiered token bucket: read 30/min, write 10/min), and a

structured error taxonomy. This transforms noted from a notebook with an AI chat into a headless AI execution engine controllable by any MCP-compatible client.

7.3 Tool System

25 tools provide read and write access to the MLOps stack: MLflow experiments and runs, Airflow DAG status and task logs, DVC tracked files, Hydra configurations, project files, Knowledge Graph entities, notebook cell navigation, web content fetching (via Camoufox anti-detect browser), and lint diagnostics. Write tools (`update_cell`, `insert_cell`, `create_file`) require explicit user confirmation with a diff preview.

A Dynamic Context Router selects only the relevant tool schemas per turn for Claude (typically 5-8 out of 25), reducing token cost while maintaining full tool coverage.

7.4 Web Content Fetch

The `fetch_url` tool retrieves web content using Camoufox, a persistent anti-detect Firefox browser with C++ level TLS fingerprint spoofing. The browser launches once as a singleton and stays warm for subsequent requests. This enables the assistant to read documentation, API references, or any URL shared by the user and incorporate the content into its analysis.

8. Demonstration Pipeline: End-to-End Workflow

The following walkthrough demonstrates the complete lifecycle within the noted platform, from configuration to prediction:

Step 1: Configuration

Open the Jena Weather project in noted. The Configuration Composer panel shows four Hydra config groups. Select model: `gru_baseline`, scaler: `standard`, training: `default` (30 epochs, batch 256, LR 0.0005, seed 42).

Step 2: Pipeline Trigger

Click “Run as Pipeline” from the notebook bar, or navigate to the Orchestration section and trigger the `jena_training_pipeline` DAG. The composed Hydra configuration is passed as DAG parameters. The 6-task pipeline begins execution.

Step 3: Live Monitoring

The pipeline status appears in noted’s status bar (blue pill). The Orchestration tree shows real-time task status updates via Socket.IO polling. Training progress is visible in the Live Metrics panel with epoch-by-epoch loss curves.

Step 4: Verification

After completion (total ~3m25s), navigate to the Experiments section. The new run appears with logged metrics (test MAE: ~2.02 C, test RMSE: ~2.56, test R2: ~0.893). The run is tagged with the DVC data hash and Hydra config hash.

Step 5: Auto-Promotion

The `promote_model` task compared the new model against the existing champion. If the test MAE improved, the model was registered in the MLflow Registry with the `@champion` alias. The Models section in the Explorer shows the registered model with its version, signature, and lineage chain.

Step 6: Prediction

Open the `jena_client` web app or noted's Try It panel. Submit a forecast request with recent climate features. The FastAPI serving container loads the `@champion` model from the registry and returns temperature predictions. The `Chart.js` frontend renders the forecast alongside historical data.

Step 7: Evidently Reports

The Evidently UI (accessible from noted's sidebar) shows two reports generated during the pipeline: a data quality report (from the `evidently_quality` task) and a drift detection report (from the `evidently_drift` task, comparing train vs test distributions). 6 of 16 features showed statistically significant drift (37.5%).

9. Conclusion

9.1 Summary

The noted platform successfully abstracts the complexity of disparate MLOps tools into a unified, production-ready workspace. The Jena Weather Forecasting project demonstrates every required component: automated Airflow pipelines, MLflow experiment tracking with model registry, FastAPI model serving, a functional web frontend, and Docker Compose containerization - all integrated within a single interface that eliminates context-switching.

The platform goes beyond the baseline requirements by providing:

- **Native AI assistant** with two LLM backends (local Gemma 4 + cloud Claude) using model-native tool calling
- **MCP Server** enabling external AI clients to control noted as a headless execution engine
- **Full reproducibility chain**: DVC data hashes + Hydra config hashes + git commits + seeds = identical results
- **Evidently integration** for data quality and drift monitoring
- **25 AI-powered tools** for workspace interaction, including web content fetch via anti-detect browser
- **Zero vendor lock-in**: all artifacts work independently of noted

9.2 Future Enhancements

Planned Phase 5 capabilities include:

- **MCP external client access:** scoped API keys with read/write permissions for external AI clients
- **MCP Resource Layer:** noted:// URI scheme for passive environment state access (MLflow metrics, DVC lineage, Airflow status)
- **KV cache persistence:** save/load llama-cpp-python KV cache state per conversation thread for faster local LLM inference
- **Evidently quality gates:** pre-pipeline data validation with automatic pipeline abort on quality threshold violation
- **Impact analysis:** Knowledge Graph-powered “What breaks if I change this?” queries
- **Inline code completion:** ghost-text suggestions via LLM integration